Reporte Técnico: Despliegue de Clúster Kubernetes Multi-Nodo en Rocky Linux 10

Materia: DevOps
Estudiante: Aldo Aaron Gonzalez Chavez
Sistema Operativo: Rocky Linux 10 (Red Quartz)
Versión de Kubernetes: v1.31.14
Container Runtime: containerd v2.4.2
Herramienta de Automatización: Ansible

---

1. Introducción y Objetivos

El presente documento detalla la implementación de un clúster de Kubernetes en topología multi-nodo (1 nodo Control Plane y 1 nodo Worker) sobre máquinas virtuales con Rocky Linux 10.

Los objetivos alcanzados comprenden:

- Automatizar la configuración de los prerrequisitos del sistema operativo mediante un Playbook declarativo de Ansible.
- Configurar una topología de red robusta basada en interfaces separadas para gestión y acceso a Internet.
- Diagnosticar y corregir fallas reales de infraestructura (corrupción de binarios, limitación de espacio en disco por LVM e inconsistencias de runtime).
- Inicializar el nodo Control Plane con `kubeadm`, desplegar la red de Pods (CNI Flannel) y unir el nodo Worker mediante autenticación criptográfica.
- Validar el estado operativo global mediante inspección del API Server y pods de sistema.

---

2. Topología de Red y Especificaciones del Entorno

Para evitar restricciones de seguridad inalámbrica (*Client Isolation*) que impiden la comunicación directa entre máquinas virtuales, se configuraron dos adaptadores de red independientes por nodo en VirtualBox:

1. **Adaptador 1 (NAT):** Provee salida a Internet para descarga de paquetes y repositorios.
2. **Adaptador 2 (Host-Only `enp0s8`):** Red privada y aislada entre el host (Windows) y los nodos Linux para administración SSH y tráfico interno del clúster (`192.168.56.0/24`).

Parámetro	Nodo Master (Control Plane)	Nodo Worker (Cómputo)
Hostname	`k8s-master`	`k8s-worker1`
IP Privada (Host-Only `enp0s8`)	`192.168.56.102`	`192.168.56.101`
IP Salida a Internet (`enp0s3`)	`10.0.2.15` (NAT)	`10.0.2.15` (NAT)

Parámetro	Nodo Master (Control Plane)	Nodo Worker (Cómputo)
Hardware Asignado	2 vCPU / 2048 MB RAM / 20 GB Disco	2 vCPU / 2048 MB RAM / 20 GB Disco
Rol en el Clúster	Control Plane (`control-plane`)	Worker (`<none>`)
Red de Pods (CIDR)	`10.244.0.0/16`	`10.244.0.0/16`

3. Automatización de Prerrequisitos con Ansible

Antes del despliegue de Kubernetes, el sistema operativo fue preparado mediante el playbook `k8s_prerequisitos.yml`. Esto garantizó que ambos nodos cumplieran los requisitos exigidos por `kubelet` de forma idempotente.

3.1 Prerrequisitos Implementados

1. **Desactivación de Swap:** Ejecución de `swapoff -a` y comentario de la partición swap en `/etc/fstab` (requerido para garantizar la precisión en el dimensionamiento de recursos de memoria).
2. **SELinux en modo Permisivo:** Modificación en `/etc/selinux/config` y aplicación inmediata con `setenforce 0` para evitar bloqueos en el sistema de archivos del contenedor.
3. **Deshabilitación de Firewall:** Parada y deshabilitación de `firewalld` para evitar el bloqueo del puerto `6443` (API Server) y `2379-2380` (etcd).
4. **Módulos del Kernel:** Carga persistente de `overlay` (OverlayFS para contenedores) y `br_netfilter` (filtrado de puentes virtuales) en `/etc/modules-load.d/k8s.conf`.
5. **Parámetros Sysctl de Red:** Configuración en `/etc/sysctl.d/k8s.conf` de:
   - `net.bridge.bridge-nf-call-iptables = 1`
   - `net.bridge.bridge-nf-call-ip6tables = 1`
   - `net.ipv4.ip_forward = 1`
6. **Container Runtime:** Añadido del repositorio oficial de Docker e instalación de `containerd.io`, activando `SystemdCgroup = true` en su configuración.
7. **Herramientas de Kubernetes:** Configuración del repositorio `pkgs.k8s.io` e instalación de `kubelet`, `kubeadm` y `kubectl`.

3.2 Evidencia de Ejecución (Ansible Play Recap)

```
PLAY RECAP *********************************************************************
nodo1                      : ok=19   changed=1    unreachable=0    failed=0
skipped=1    rescued=0    ignored=0
```

---

4. Resolución de Incidencias Técnicas Durante el Despliegue

Durante la preparación de los nodos se diagnosticaron y solucionaron tres problemas a nivel de sistema operativo:

Incidencia 1: Corrupción de Binarios tras Corte de Red

- **Diagnóstico:** El comando `kubeadm` no generaba salida y su tamaño era de `0 bytes`. Posteriormente, la herramienta de inspección `crictl` arrojaba `Violación de segmento` (*missing section headers at 30062736*).
- **Causa:** La máquina virtual se detuvo a mitad de una transacción de DNF, dejando los ejecutables corruptos en disco.
- **Solución Ejecutada:**

```
dnf reinstall -y kubelet kubeadm kubectl cri-tools --
disableexcludes=kubernetes
```

Se verificó la integridad del ejecutable restaurado a `57 MB` y `crictl version v1.31.1`.

Incidencia 2: Partición `/var` Saturada (`no space left on device`)

- **Diagnóstico:** Al intentar descargar las imágenes del clúster con `kubeadm init`, fallaba la extracción de capas de `kube-scheduler`, `coredns` y `etcd` con el error `write: no space left on device`.
- **Causa:** El particionado automático por LVM de Rocky Linux asignó un volumen lógico de únicamente `960 MB` para `/var` (que se saturó al 93%), mientras que la partición raíz `/` contaba con más de `12 GB` disponibles.
- **Solución Ejecutada:** Se purgó el almacenamiento residual y se reubicó la raíz de trabajo de containerd mediante un enlace simbólico hacia la partición principal:

```
mkdir -p /container-data
rm -rf /var/lib/containerd
ln -s /container-data /var/lib/containerd
systemctl restart containerd
```

La partición raíz quedó con `12 GB` disponibles (`16%` de uso) y `/var` liberado al `61%`.

---

5. Procedimiento de Despliegue Paso a Paso

A continuación se detalla la secuencia de comandos ejecutados en cada terminal.

Paso 1: Configuración de Hostname y Resolución Local

En Terminal Master (`ssh root@192.168.56.102`):

```
hostnamectl set-hostname k8s-master
cat << 'EOF' >> /etc/hosts
192.168.56.102 k8s-master
192.168.56.101 k8s-worker1
EOF
```

En Terminal Worker (`ssh root@192.168.56.101`):

```
hostnamectl set-hostname k8s-worker1
cat << 'EOF' >> /etc/hosts
192.168.56.102 k8s-master
192.168.56.101 k8s-worker1
EOF
```

Paso 2: Inicialización del Plano de Control (Master)

En Terminal Master (`ssh root@192.168.56.102`):

```
kubeadm init --apiserver-advertise-address=192.168.56.102 --pod-network-
cidr=10.244.0.0/16
```

Salida de Terminal Obtenida:

```
[init] Using Kubernetes version: v1.31.14
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[control-plane] Creating static Pod manifest for "kube-apiserver"
[control-plane] Creating static Pod manifest for "kube-controller-manager"
[control-plane] Creating static Pod manifest for "kube-scheduler"
[kubelet-start] Starting the kubelet
[wait-control-plane] Waiting for the kubelet to boot up the control plane as
static Pods
[api-check] The API server is healthy after 1m55.503925149s
[upload-config] Storing the configuration used in ConfigMap "kubeadm-config" in
"kube-system"
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Then you can join any number of worker nodes by running the following on each as
root:

kubeadm join 192.168.56.102:6443 --token 1xvcr4.g0qhg3txygyyawa5 \
        --discovery-token-ca-cert-hash
sha256:90e1f0c0593a8f7a9bf0e1e743886617c7d88e6fdf78c5dfde736fceda360403
```

Paso 3: Configuración de Kubeconfig y Red CNI (Master)

En Terminal Master (`ssh root@192.168.56.102`):

```
mkdir -p $HOME/.kube
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
chown $(id -u):$(id -g) $HOME/.kube/config

kubectl apply -f https://github.com/flannel-
io/flannel/releases/latest/download/kube-flannel.yml
```

Salida de Terminal Obtenida:

```
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
```

Paso 4: Unión del Nodo Worker al Clúster (Worker)

En Terminal Worker (`ssh root@192.168.56.101`):

```
kubeadm join 192.168.56.102:6443 --token 1xvcr4.g0qhg3txygyyawa5 \
        --discovery-token-ca-cert-hash
sha256:90e1f0c0593a8f7a9bf0e1e743886617c7d88e6fdf78c5dfde736fceda360403
```

Salida de Terminal Obtenida:

```
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get
cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file
"/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file
"/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz.
This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.01118864s
```

```
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

---

6. Verificación y Evidencias de Operatividad

Las siguientes consultas se ejecutaron desde el nodo Master para certificar que el clúster se encuentra en estado completamente saludable y operativo.

6.1 Inspección del Estado de los Nodos

Ambos nodos reportan el estado `Ready`, lo que certifica que el runtime CRI y el plugin de red CNI Flannel están sincronizados correctamente:

```
[root@k8s-master ~]# kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE     VERSION    INTERNAL-IP
EXTERNAL-IP   OS-IMAGE                        KERNEL-VERSION
CONTAINER-RUNTIME
k8s-master    Ready    control-plane   11m     v1.31.14   192.168.56.102   <none>
Rocky Linux 10.2 (Red Quartz)   6.12.0-211.16.1.el10_2.0.1.x86_64
containerd://2.4.2
k8s-worker1   Ready    <none>          5m30s   v1.31.14   192.168.56.101   <none>
Rocky Linux 10.2 (Red Quartz)   6.12.0-211.16.1.el10_2.0.1.x86_64
containerd://2.4.2
```

6.2 Inspección Global de Pods del Sistema

Todos los Pods en los namespaces del sistema (`kube-system` y `kube-flannel`) alcanzaron el estado `Running (1/1)` con 0 reinicios:

```
[root@k8s-master ~]# kubectl get pods -A
NAMESPACE      NAME                                READY   STATUS    RESTARTS
AGE
kube-flannel   kube-flannel-ds-f7l47               1/1     Running   0
5m35s
kube-flannel   kube-flannel-ds-plkwh               1/1     Running   0
7m43s
kube-system    coredns-7c65d6cfc9-dh9t4            1/1     Running   0
9m42s
kube-system    coredns-7c65d6cfc9-qgrxx            1/1     Running   0
9m42s
kube-system    etcd-k8s-master                     1/1     Running   0
9m50s
```

```
kube-system    kube-apiserver-k8s-master            1/1     Running   0
9m50s
kube-system    kube-controller-manager-k8s-master   1/1     Running   0
9m50s
kube-system    kube-proxy-9n2sl                     1/1     Running   0
5m35s
kube-system    kube-proxy-t4pp6                     1/1     Running   0
9m43s
kube-system    kube-scheduler-k8s-master            1/1     Running   0
9m50s
```

---

7. Procedimiento de Apagado y Encendido del Clúster

En Kubernetes, la configuración y el estado del clúster (certificados, configuración de red, registros de etcd y pertenencia de nodos) quedan grabados de forma persistente en disco. Por lo tanto, **no es necesario volver a ejecutar `kubeadm init` ni `kubeadm join` al reiniciar las máquinas**.

7.1 Procedimiento de Apagado Ordenado

Para evitar corrupción en la base de datos de `etcd`, se recomienda apagar primero los nodos de cómputo (Worker) y finalmente el nodo Control Plane (Master).

1. En Terminal Worker (`ssh root@192.168.56.101`):

```
poweroff
```

2. En Terminal Master (`ssh root@192.168.56.102`):

```
poweroff
```

7.2 Procedimiento de Encendido y Reactivación

Al arrancar nuevamente las máquinas virtuales en VirtualBox:

1. Orden de Inicio:

- Iniciar primero la máquina virtual del **Master** (`Rocky-Linux-DevOps`) en VirtualBox.
- Iniciar después la máquina virtual del **Worker** (`Rocky-Linux-DevOps-worker`).

2. Verificación Automática: Los servicios de sistema `containerd` y `kubelet` están habilitados en el arranque (`systemctl enable`), por lo que arrancan de forma autónoma. Simplemente conéctate por SSH al Master y valida el estado:

En Terminal Master (`ssh root@192.168.56.102`):

```
kubectl get nodes
```

(Durante el primer minuto tras el arranque, los nodos pueden figurar en `NotReady` mientras se sincroniza el API Server y Flannel; tras 30 a 60 segundos pasarán automáticamente a `Ready`).

3. Protocolo en caso de que algún servicio no levante: Si tras un par de minutos algún nodo sigue en `NotReady`, ejecuta en dicho nodo:

```
systemctl restart containerd kubelet
```

---

8. Conclusiones

1. **Gestión Declarativa de Infraestructura:** La implementación de Ansible demostró ser la metodología adecuada para garantizar la idempotencia de prerrequisitos del kernel y del sistema operativo en entornos RHEL/Rocky Linux.
2. **Diagnóstico Integral:** Se demostró la importancia de analizar los logs del kernel, la integridad de los paquetes y la distribución de volúmenes lógicos LVM, evitando diagnósticos erróneos cuando un servicio falla en el aprovisionamiento.
3. **Validación del Clúster:** Se consolidó una plataforma de Kubernetes multi-nodo estable y funcional, con DNS interno (CoreDNS) y enrutamiento overlay operativo, lista para el despliegue de cargas de trabajo de producción.